



Pontificia Universidad Católica de Chile
Escuela de Ingeniería
Departamento de Ciencia de la Computación

Clase 27: Patrones de Comportamiento (parte 2)

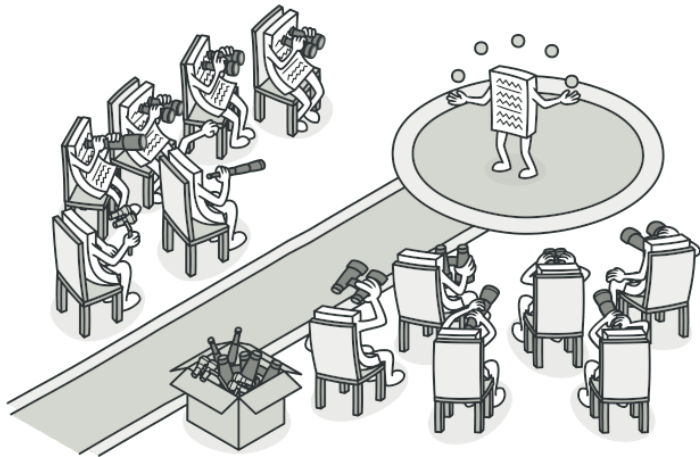
Francisco Gazitúa Requena (fco_gazitua_requena@uc.cl)

IIC2113 Diseño Detallado de Software

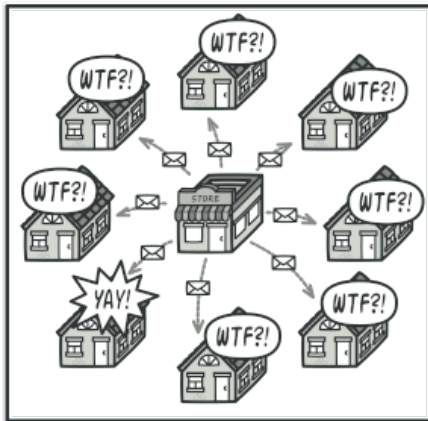
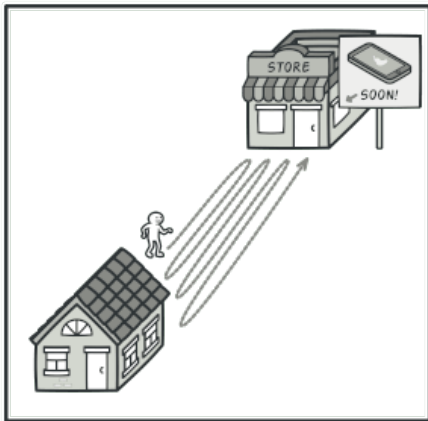
11 de Noviembre, 2025

Observer

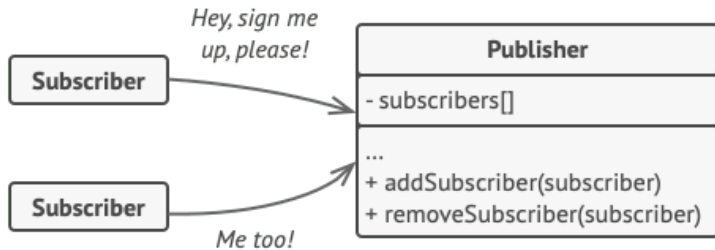
Observer



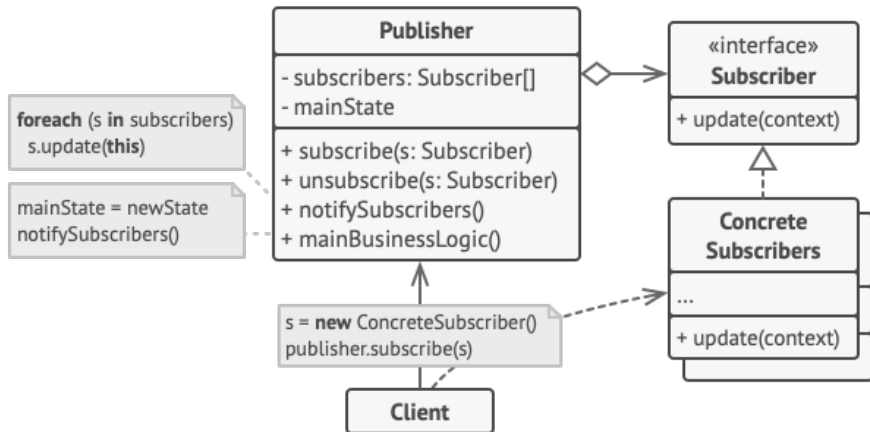
Observer



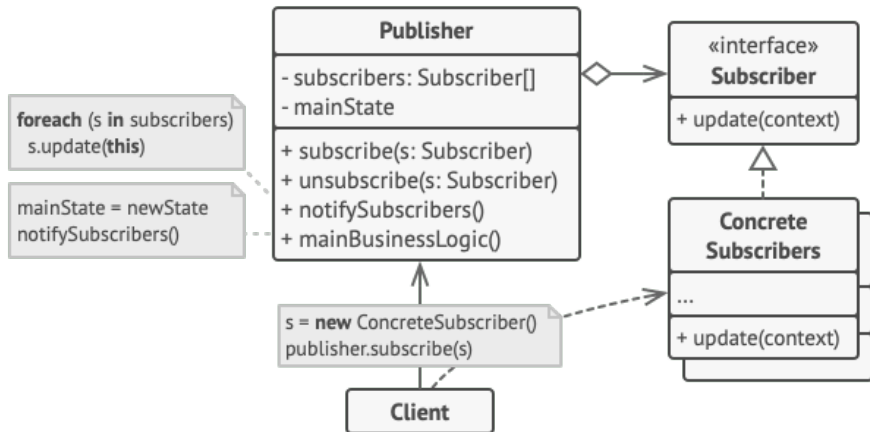
Observer



Observer

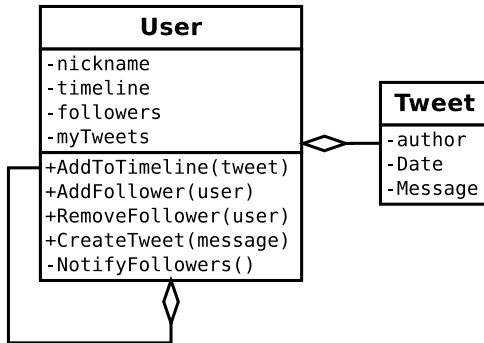


Observer

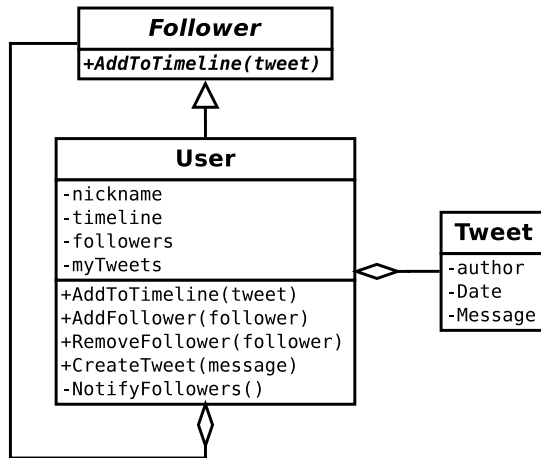


Veamos un ejemplo

Observer



Observer

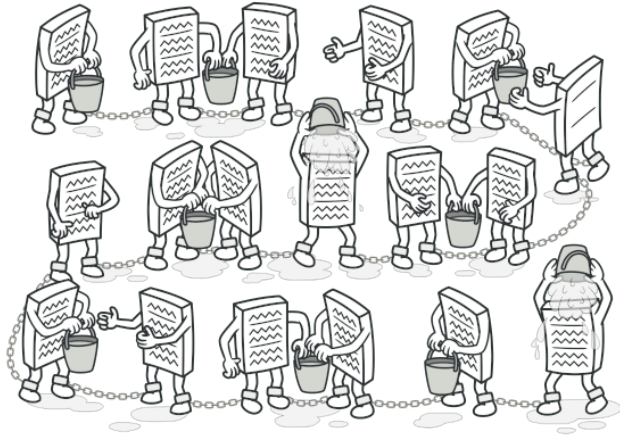


Observer

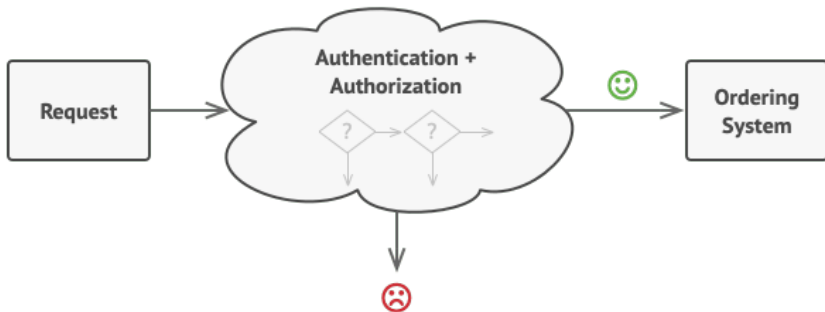
```
2 public class User : Follower {
3     /* ... */
4     public void AddToTimeline(Tweet tweet) => timeline.Add(tweet);
5
6     public void AddFollower(Follower newFollower)
7         => followers.Add(newFollower);
8     public void RemoveFollower(Follower follower)
9         => followers.Remove(follower);
10
11     public void AddNewTweet(string message) {
12         Tweet tweet = new Tweet(nickname, message);
13         myTweets.Add(tweet);
14         NotifyFollowers(tweet);
15     }
16     private void NotifyFollowers(Tweet tweet) {
17         foreach (var follower in followers)
18             follower.AddToTimeline(tweet);
19     }
20 }
```

Chain of Responsibility

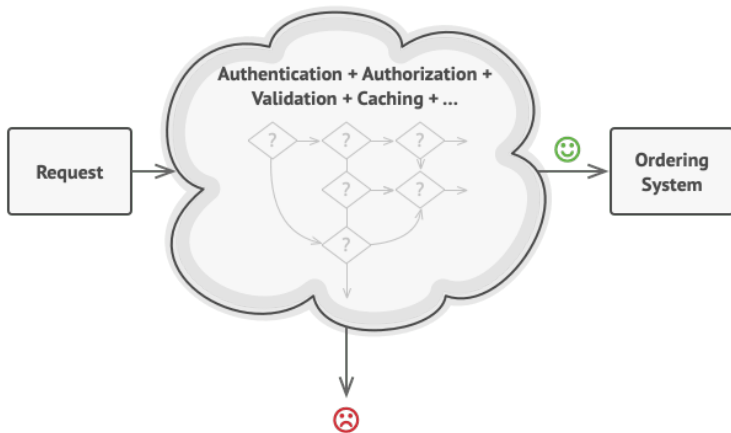
Chain of Responsibility



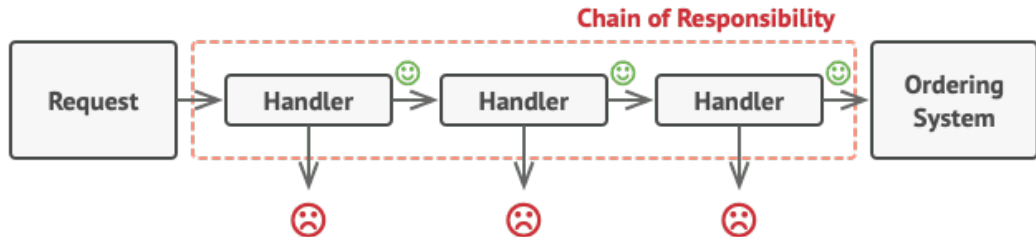
Chain of Responsibility



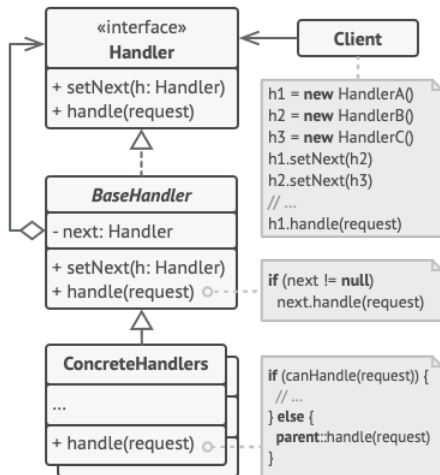
Chain of Responsibility



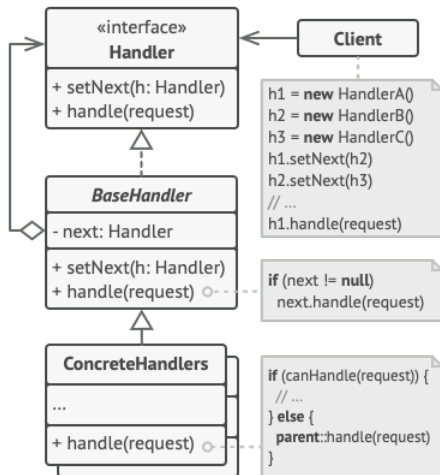
Chain of Responsibility



Chain of Responsibility



Chain of Responsibility



Veamos un ejemplo

Chain of Responsibility

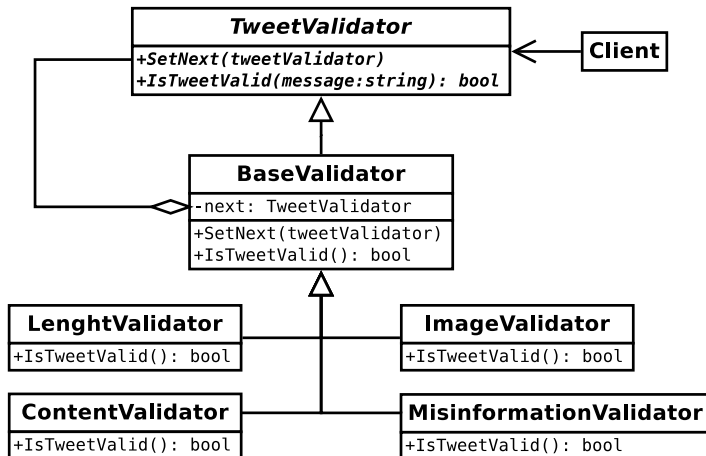
```
2 public bool IsTweetValid(string message){
3     if (IsTheLenghtCorrect(message))
4         return false;
5
6     if (DoesItDiscussInappropriateTopics(message))
7         return false;
8
9     if (DoesItContainInappropriateImages(message))
10        return false;
11
12    if (DoesItContainMisinformation(message))
13        return false;
14
15    /* ... */
16    return true;
17 }
```

Chain of Responsibility

```
2 public bool IsTweetValid(string message){
3     if (IsTheLenghtCorrect(message))
4         return false;
5
6     if (DoesItDiscussInappropriateTopics(message))
7         return false;
8
9     if (DoesItContainInappropriateImages(message))
10        return false;
11
12    if (DoesItContainMisinformation(message))
13        return false;
14
15    /* ... */
16    return true;
17 }
```

¡Esta solución rompe el OCP!

Chain of Responsibility



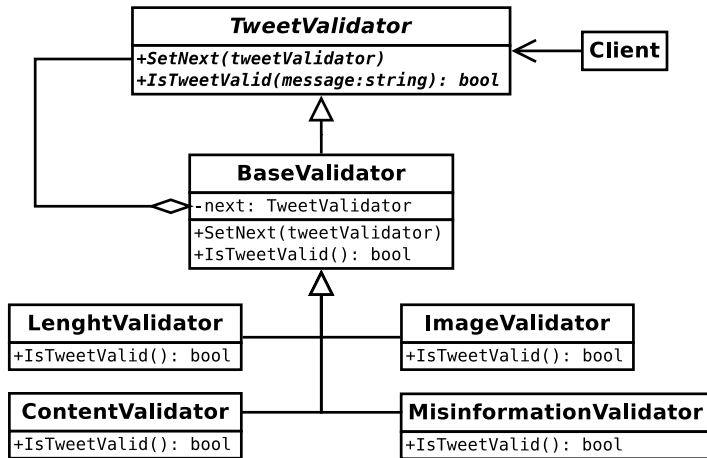
Chain of Responsibility

```
2 public interface TweetValidator {
3     void SetNext(TweetValidator tweetValidator);
4     bool IsTweetValid(string message);
5 }
6
7 public class BaseValidator : TweetValidator
8 {
9     private TweetValidator? next = null;
10
11     public void SetNext(TweetValidator tweetValidator)
12         => next = tweetValidator;
13
14     public virtual bool IsTweetValid(string message) {
15         if (next != null)
16             return next.IsTweetValid(message);
17         return true;
18     }
19 }
```

Chain of Responsibility

```
21 public class LenghtValidator:BaseValidator {  
22     private const int MaxLenghtTweet = 280;  
23  
24     public override bool IsTweetValid(string message) {  
25         if(message.Length <= MaxLenghtTweet)  
26             return base.IsTweetValid(message);  
27         return false;  
28     }  
29 }
```

Chain of Responsibility



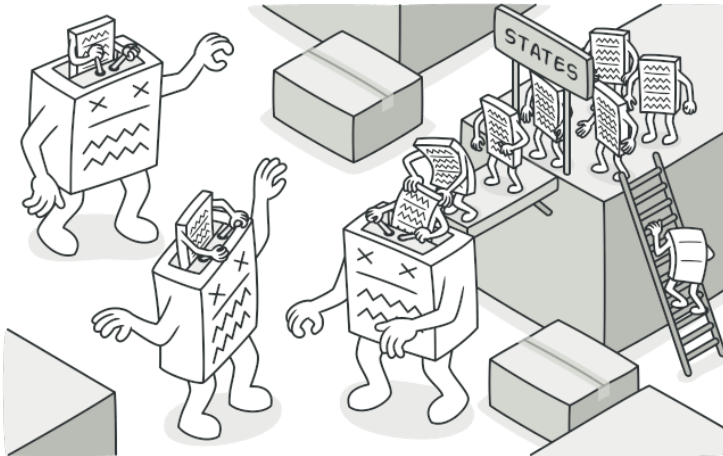
Pregunta tipo examen: Dado todo este código, crea un validador que chequee todo menos si el tweet contiene *misinformation*.

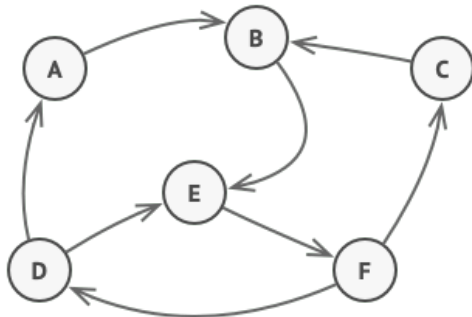
Chain of Responsibility

```
31 public bool ValidateTweet(string message) {  
32     TweetValidator validator = new LenghtValidator();  
33     TweetValidator imageValidator = new ImageValidator();  
34     TweetValidator contentValidator = new ContentValidator();  
35     validator.SetNext(imageValidator);  
36     imageValidator.SetNext(contentValidator);  
37  
38     return validator.IsTweetValid(message);  
39 }
```


State

State

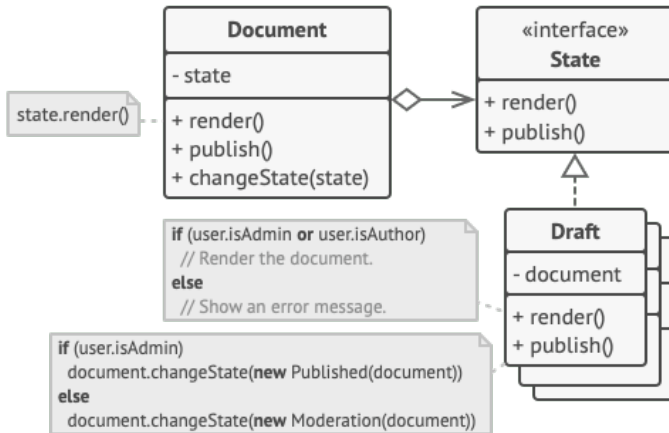




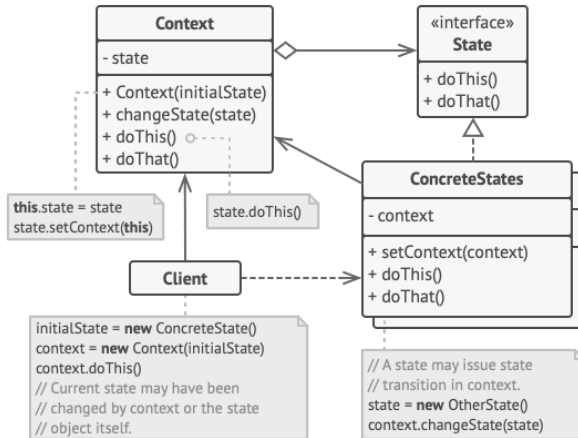
State



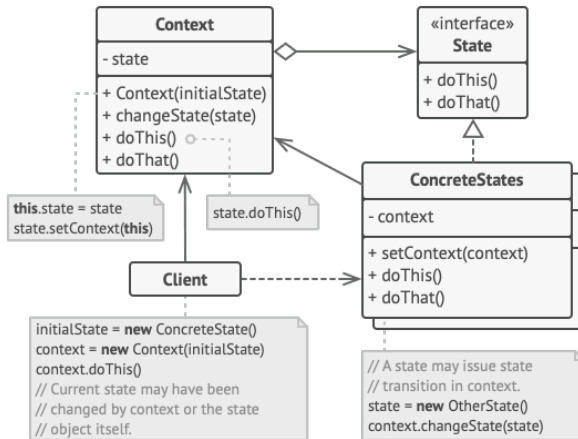
State



State

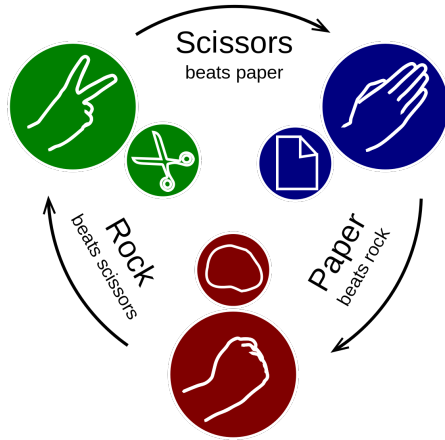


State



Veamos un ejemplo

State



State

Interfaz que representa al estado:

```
1 public interface IState {  
2     Play GetNextPlay();  
3     IState GetNextState();  
4 }
```

State

Interfaz que representa al estado:

```
1 public interface IState {  
2     Play GetNextPlay();  
3     IState GetNextState();  
4 }
```

Estados concretos:

```
1 public class Rock: IState {  
2     public Play GetNextPlay()  
3         => Play.Rock;  
4  
5     public IState GetNextState()  
6         => new Paper();  
7 }
```

State

Interfaz que representa al estado:

```
1 public interface IState {  
2     Play GetNextPlay();  
3     IState GetNextState();  
4 }
```

Estados concretos:

```
1 public class Paper: IState {  
2     public Play GetNextPlay()  
3         => Play.Paper;  
4  
5     public IState GetNextState()  
6         => new Scissors();  
7 }
```

State

Interfaz que representa al estado:

```
1 public interface IState {  
2     Play GetNextPlay();  
3     IState GetNextState();  
4 }
```

Estados concretos:

```
1 public class Scissors: IState {  
2     public Play GetNextPlay()  
3         => Play.Scissors;  
4  
5     public IState GetNextState()  
6         => new Rock();  
7 }
```

State

Interfaz que representa al estado:

```
1 public interface IState {  
2     Play GetNextPlay();  
3     IState GetNextState();  
4 }
```

Implementa una AI juegue: piedra, papel, tijeras, piedra, papel, tijeras, piedra ... etc.

```
1 public class AI {  
2     public Play GetNextPlay() {  
3         // TODO: Crea la lógica de la IA  
4         throw new NotImplementedException();  
5     }  
6 }
```

State

Interfaz que representa al estado:

```
1 public interface IState {  
2     Play GetNextPlay();  
3     IState GetNextState();  
4 }
```

Implementa una AI juegue: piedra, papel, tijeras, piedra, papel, tijeras, piedra ... etc.

```
1 public class AI {  
2     private IState _state = new Scissors();  
3  
4     public Play GetNextPlay() {  
5         _state = _state.GetNextState();  
6         return _state.GetNextPlay();  
7     }  
8 }
```

¿Qué ganamos con esta forma de resolver el problema?

¿Qué ganamos con esta forma de resolver el problema?

```
1 public class AI {  
2     private Play _currentPlay = Play.Scissors;  
3  
4     public Play GetNextPlay() {  
5         if(_currentPlay == Play.Rock)  
6             _currentPlay = Play.Paper;  
7         else if(_currentPlay == Play.Paper)  
8             _currentPlay = Play.Scissors;  
9         else if(_currentPlay == Play.Scissors)  
10            _currentPlay = Play.Rock;  
11  
12         return _currentPlay;  
13     }  
14 }
```

... nos ahorramos esta cadena de ifs que en un programa serio podría ser una tortura.